

IoT-Core-DynamoDB-Hands-on Step-by-Step Guide

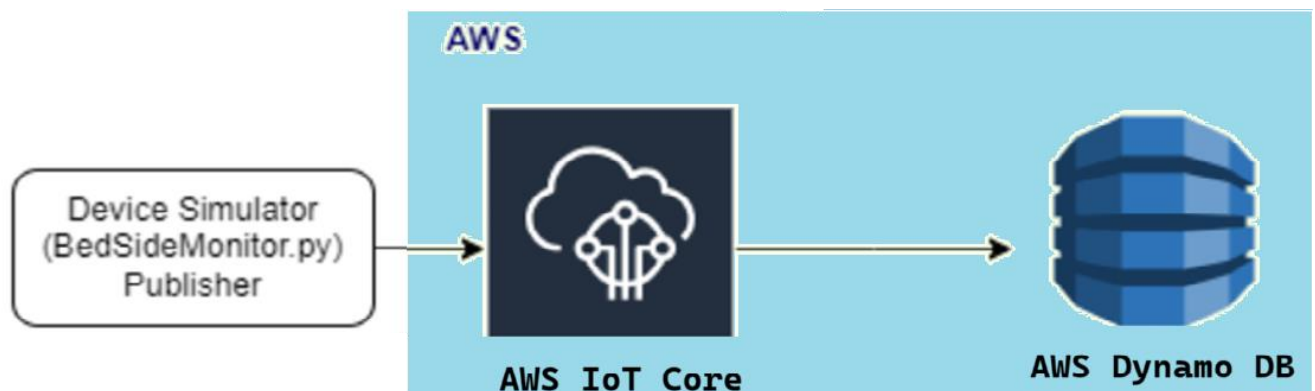
AWS IoT Core

In this hands-on, we are going to go through a complete workflow to pushing and storing data in the cloud. a Using healthcare device - Bed side monitor we would learn to do following in this exercise -

- Create an IoT thing and related entities - type and groups.
- Create and download the client certificate and keys, and also set up the right policy to allow authorization
- Configure the local simulator program based on the certificate and keys, transmit and validate the data using the MQTT test client on AWS IoT Core.
- Create a DynamoDB table and create an IoT core rule to automatically route incoming data to that table.

Note: - You can do all of these in your AWS Free tier account.

high level flow diagram



Entities in IoT Core

The basic building block is a IoT Thing. A Thing can be an end device generating sensor data using one or more sensors. Edge devices using Greengrass SDK can be configured as a Greengrass core device. While adding a device to the thing registry, you can setup the following properties:

- **Name:** Is a unique string which identifies that device across your whole system. Normally, companies will come up with an internal nomenclature to indicate the device type and id, thereby making it unique. Let's say we add a device called 'BSM_A101'
- **Type:** Each device can be associated with a Type which can be created beforehand. Each Type can have three searchable attributes. When you add a type to a device, you can set the value of each attribute.
- **Groups:** Another way of aggregating devices is by their topology i.e. Groups. One can create hierarchical groups and then map all devices to them based on your system topology. This is very

Proprietary content. © RKS Tech (www.rkstech.in).

All Rights Reserved. Unauthorized use or distribution prohibited.



useful to target a group of devices based on some criteria. Each group can have its own set of attributes as well. Same like Type, you can create a Group beforehand too.

We can also map other groups to the same device. For ex. we can think of another set of group hierarchy based on power consumption, communication protocol, or even different types of lights.

There is even a possibility of creating a dynamic group where the group membership is defined by a query defined by some attribute values of the devices. For ex. you can create a dynamic group which only has member devices for which the temperature reading is more than 80 degrees. This functionality can be used to highlight and target a group based on dynamic parameters. This is similar to creating a dynamic group of servers where memory usage has crossed 80% limit

- **Thing shadow:** In addition, you can also set the initial parameters that should be part of the device shadow. We'll go into details of device shadows later in the notebook.

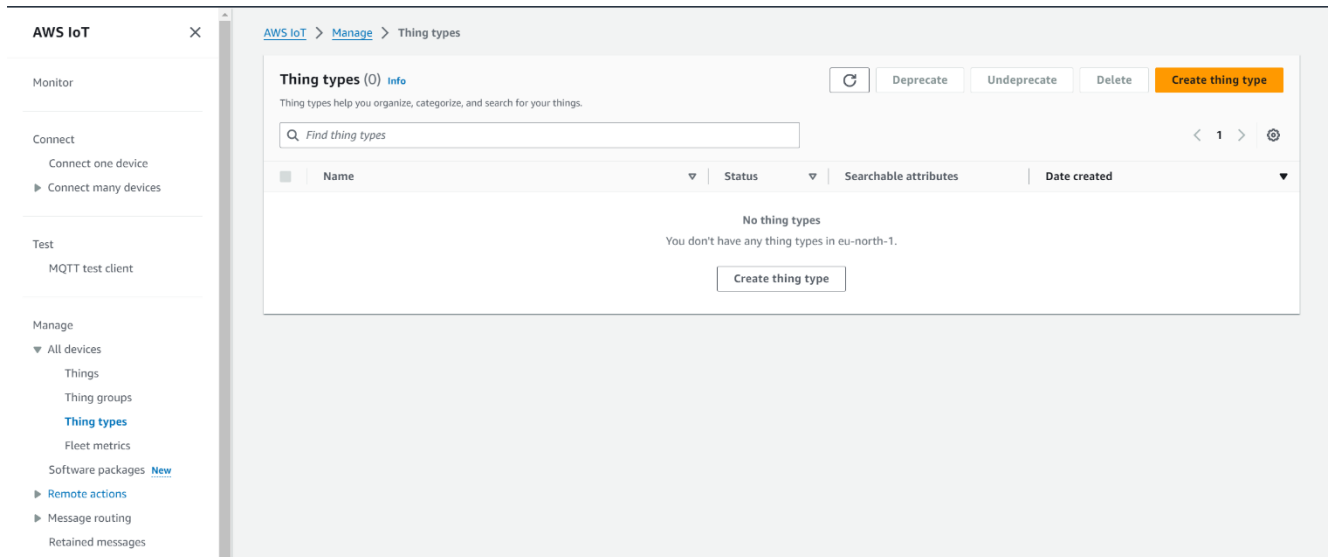


AWS IoT Core: Group & Type Creation

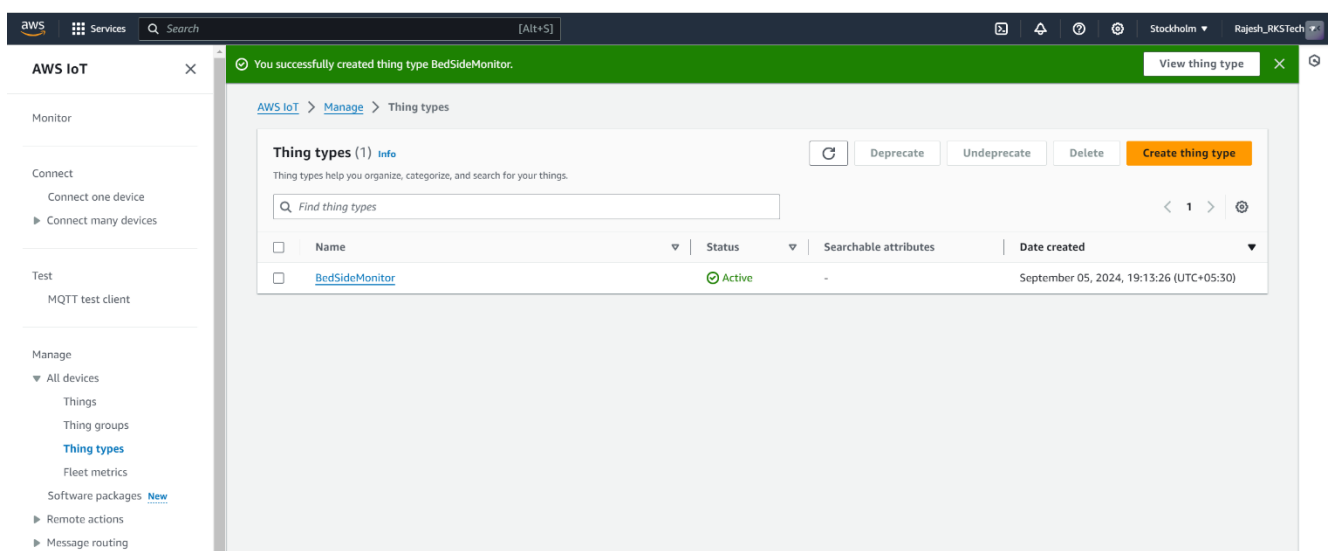
In this section, we are going to create Thing Group & Thing Type on AWS IoT core for our IoT thing. Data will be collected from the Bed side monitor simulator.

Type Creation

1. Go to AWS IoT core and click on Manage tab and choose Types. Click on Create thing type.



2. After clicking, it will take you to the Type creation page. Here you need to add details related to your thing type – like type name (BedSideMonitor or BSM_01 or any suitable name) and description (optional). We can ignore to add searchable attributes and tag as of now. If the operation is successful, you will see the newly created Thing Type listed.

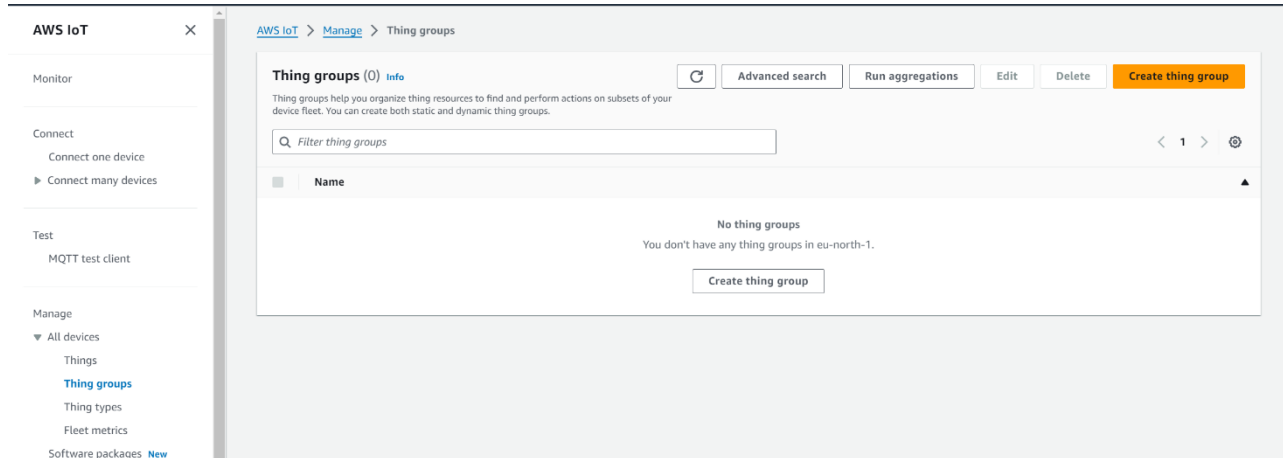


Thing Type is created for your IoT thing. Next step is to create Thing Groups.

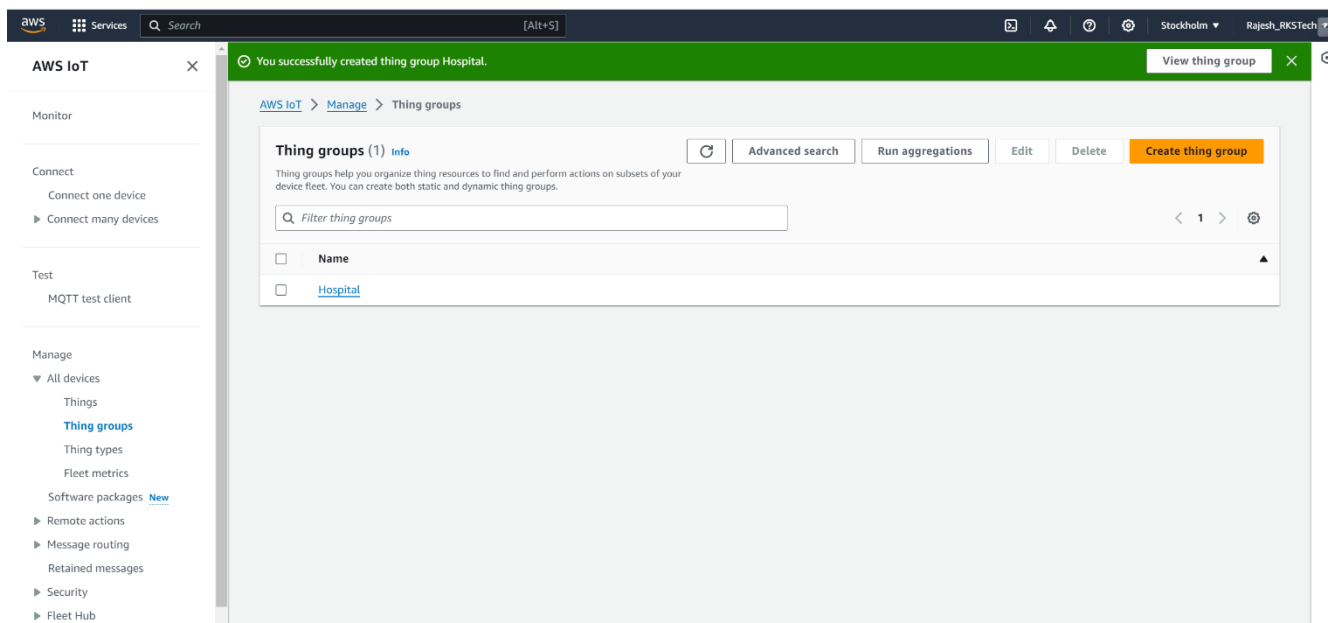
Group Creation

In this section we are going to see step by step guidance for Thing Group creation on AWS IoT core.

1. Go to AWS IoT core and click on Manage tab and choose Thing groups. Click on Create thing group.



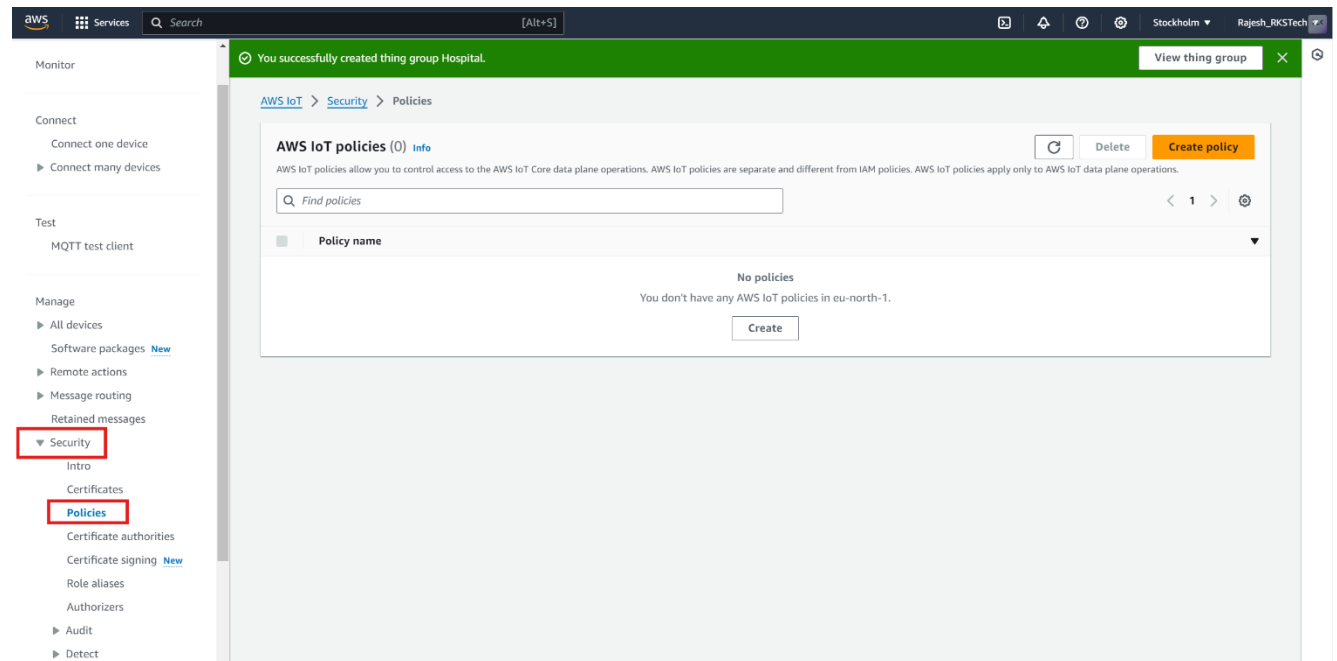
2. Click on Create a static thing group. Enter the name of the group. (eg 'Hospital') . It represents here that IoT thing is going to be present in a Hospital. You can provide group name as per your choice. You can also create hierarchical groups by creating Hospital first, and then creating Department_01 as a sub-group within that. Enter your group name and group attribute information (optional).
3. Once the details are provided, click on the Create thing group. Successful operation will take you to the next step where you can see the information related to the newly created group.



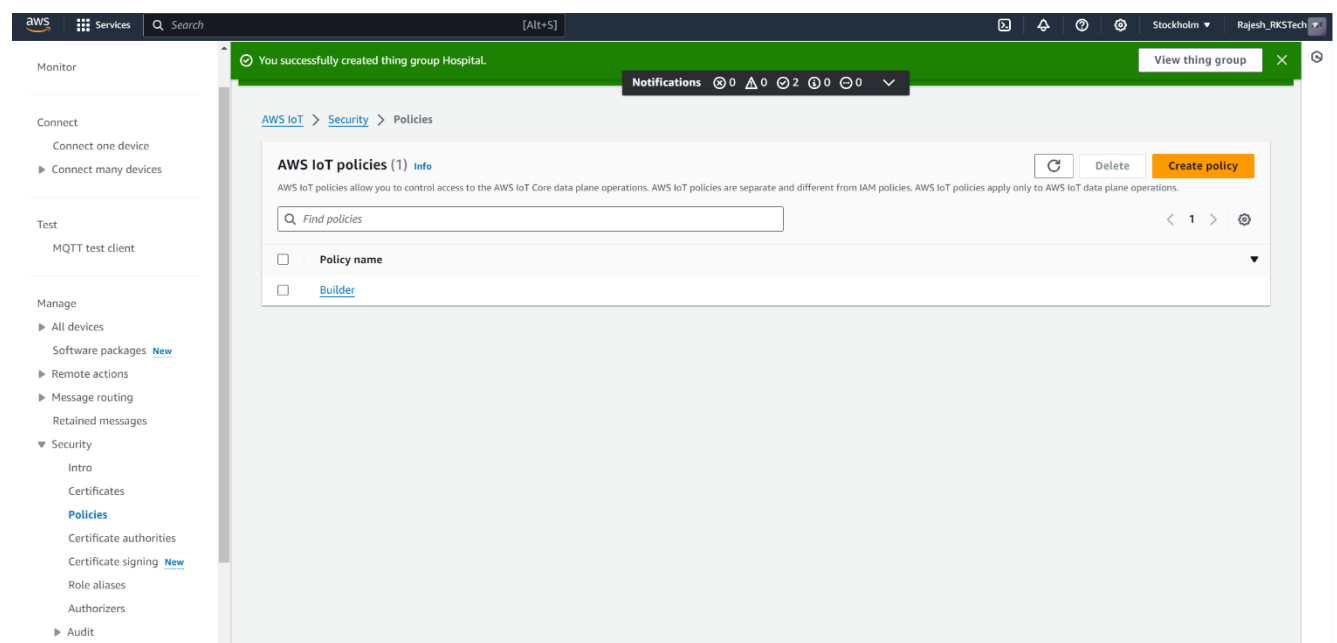
AWS IoT Core : Policy Creation

Every device in AWS IoT Core needs to have certain policies attached with them to perform any operation. We are going to create Policy for our things and set the topic name. The same topic name will be used again in the MQTT test client to see the pushed data on AWS IoT core.

1. Click on Secure and choose Policies. Click on the Create Policy and this will prompt you to the next page where you can create your own policy.



2. Enter the name for your policy.
Let the Policy document be set to Builder. Choose Allow option for Policy effect
Choose All AWS IoT actions (*) in the Policy action.
For Policy resource enter *





This will look as mentioned in the attached image. We have essentially given all permissions on all resources in this policy.

The screenshot shows the AWS IoT Builder console. The left sidebar contains navigation options: Monitor, Connect, Test, and Manage. The main content area is titled 'Builder' and shows details for a policy named 'Builder'. The 'Details' section includes the Policy ARN, Active version (1), Created date, and Last updated date. The 'Active version: 1' section shows a table with columns for Policy effect, Policy action, and Policy resource. The table contains one row with 'Allow' as the effect, '*' as the action, and '*' as the resource. The 'All versions (1)' section shows a table with columns for Version number, Status, and Created date. The table contains one row with version '1', status 'Active', and created date 'September 05, 2024, 19:20:16 (UTC+05:30)'.

AWS IoT Core : Thing Creation

- We have created Types, Groups & Policy for our thing.
- Now we should create our first thing in AWS IoT core.

1. Go to AWS IoT core and click on Manage and then choose Things. Click on Create things.

The screenshot shows the AWS IoT Core console. The left sidebar contains navigation options: Monitor, Connect, Test, and Manage. The main content area is titled 'Things' and shows a table with columns for Name and Thing type. The table is empty, and a 'Create things' button is visible. The page also includes a search bar and a 'Filter things by: name, type, group, billing, or searchable attribute' dropdown.

2. Click on Create a single thing. (Choose the first option). Enter the details for your thing such as Thing name (this is the equivalent of device id) and choose Thing type and add it into the Thing groups we created in previous steps.





3. We need to attach the policy that we created in the previous step. Select the policy and click on Create thing. Once the policy is attached with the thing that we created a new window will open
4. Now we'll create the certificates for thing. Click on the recommended option here to create the files in the next step.

AWS IoT > Manage > Things > Create things > Create single thing

Step 1
[Specify thing properties](#)

Step 2 - optional
Configure device certificate

Step 3 - optional
[Attach policies to certificate](#)

Configure device certificate - *optional* info

A device requires a certificate to connect to AWS IoT. You can choose how to register a certificate for your device now, or you can create and register a certificate for your device later. Your device won't be able to connect to AWS IoT until it has an active certificate with an appropriate policy.

Device certificate

- ☒ **Auto-generate a new certificate (recommended)**
Generate a certificate, public key, and private key using AWS IoT's certificate authority.
- ☐ **Use my certificate**
Use a certificate signed by your own certificate authority.
- ☐ **Upload CSR**
Register your CA and use your own certificates on one or many devices.
- ☐ **Skip creating a certificate at this time**
You can create a certificate for this thing and attach a policy to the certificate at a later time.

Cancel Previous **Next**

6. Make sure you download all the certificates such as Device certificate, Public key file, Private key file and Root certificates. (We will need these later).
7. Once all certificates are downloaded, then click on Done in the above step. Successful operation will create the Thing for you as follows-





Download certificates and keys

Download certificate and key files to install on your device so that it can connect to AWS.

Device certificate

You can activate the certificate now, or later. The certificate must be active for a device to connect to AWS IoT.

Device certificate
8c7ece80959...te.pem.crt

[Deactivate certificate](#) [Download](#)

Key files

The key files are unique to this certificate and can't be downloaded after you leave this page. Download them now and save them in a secure place.

⚠ This is the only time you can download the key files for this certificate.

Public key file
8c7ece8095925df759fe9f4...3ead04a-public.pem.key [Download](#)

Private key file
8c7ece8095925df759fe9f4...ead04a-private.pem.key [Download](#)

Root CA certificates

Download the root CA certificate file that corresponds to the type of data endpoint and cipher suite you're using. You can also download the root CA certificates later.

Amazon trust services endpoint
RSA 2048 bit key: Amazon Root CA 1 [Download](#)

Amazon trust services endpoint
ECC 256 bit key: Amazon Root CA 3 [Download](#)

If you don't see the root CA certificate that you need here, AWS IoT supports additional root CA certificates. These root CA certificates and others are available in our developer guides. [Learn more](#)

[Done](#)

aws Services Search [Alt+S] Stockholm Rajesh_RKSTech

AWS IoT You successfully created thing group Hospital. View thing group

Notifications 0 0 4 0 0

[AWS IoT](#) > [Manage](#) > Things

Things (1) Info [Refresh](#) [Advanced search](#) [Run aggregations](#) [Edit](#) [Delete](#) [Create things](#)

An IoT thing is a representation and record of your physical device in the cloud. A physical device needs a thing record in order to work with AWS IoT.

Filter things by: name, type, group, billing, or searchable attribute.

☐	Name	Thing type
☐	BSM_01	BedSideMonitor



Security: Certificates, Keys and Policies

IoT Core provides a comprehensive infrastructure for authentication and authorization.

- **Server Authentication:** When your devices connect to an endpoint on IoT Core, the device needs to be sure that it's connecting to the right server. AWS provides a root CA certificate that can be used to confirm this. This works similar to how a normal website https access works.
- **Client Authentication:** When creating a device, AWS can create a certificate and public/private key pair to enable authentication of the device. It's recommended to use one certificate per device. These are used when connecting to the IoT Core endpoint so that the server can verify the client's authenticity.
- **Authorization:** Authentication just confirms that the device communication is authentic and is mapped to a Thing created in IoT Core. It is still not allowed to do any actions without authorization. Authorization is provided by creating policies and attaching them to the client certificates. One policy can be mapped to multiple device certificates and hence multiple devices.

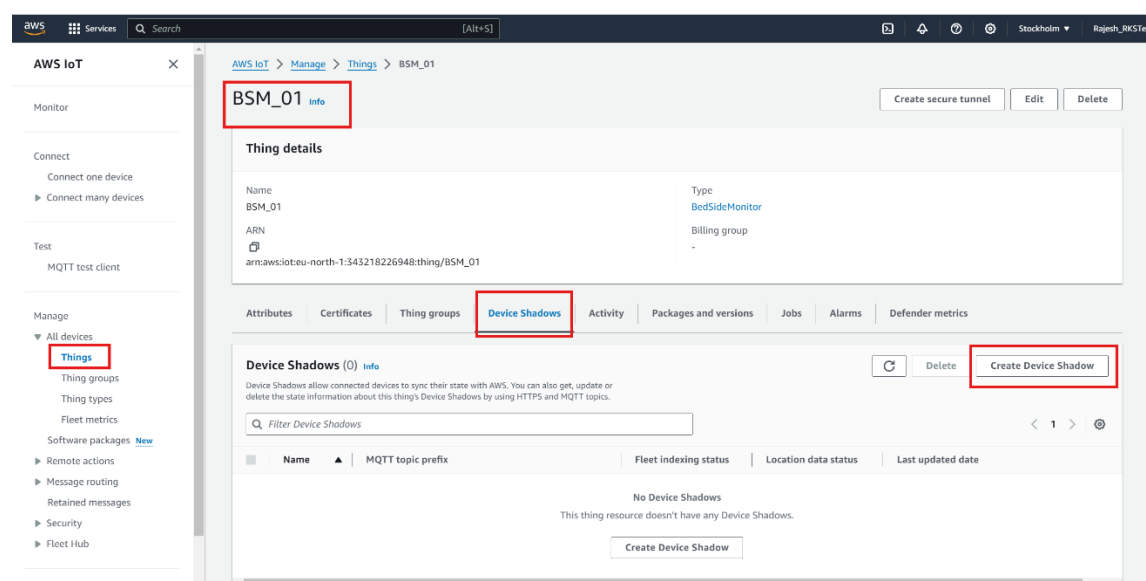
Device Shadows

Device Shadow is a concept to create a status map of a device. Different types of shadows can be created based on device types. Each shadow has a set of parameters which the device is supposed to provide. Each of these can have a desired state defined in addition to the current reported status.

In addition to the normal data flow, devices have another communication path to regularly update their device shadows. These cached parameters called shadows can be used by any upstream service to get the device status without actively connecting to the device.

There are specific actions associated with shadow updates that need to be enabled in policies. In addition, these updates are published to a separate structured MQTT topic which is automatically mapped to the shadow.

1. To create a Device Shadow, click the Thing we previously created and go to Device Shadow tab. You will see a screen as follows-





- Click on Create Shadow, a new window will open as shown below. Give the shadow name and click on Create button.

AWS IoT > Manage > Things > BSM_01

BSM_01 [Info](#)

Create secure tunnel Edit Delete

Thing details

Name BSM_01	Type BedSideMonitor
ARN arn:aws:iot:eu-north-1:343218226948:thing/BSM_01	Billing group -

Attributes Certificates Thing groups **Device Shadows** Activity 1 Packages and versions Jobs Alarms Defender metrics

Device Shadows (1) [Info](#)

Device Shadows allow connected devices to sync their state with AWS. You can also get, update or delete the state information about this thing's Device Shadows by using HTTPS and MQTT topics.

< 1 >

☐	Name ▲	MQTT topic prefix	Fleet indexing status	Location data status	Last updated date
☐	BSM_01	\$aws/things/BSM_01/shadow/name/BSM_01	Not selected	Not selected	September 05, 2024, 19:31:23 (UTC+05:30)





AWS IoT Core : MQTT Test Client

MQTT Test Client: Command

- Till above steps an IoT thing for the IoT types and group is created.
- Next step is to run the code of Bedside Monitor (Device Simulator) to push data in IoT Core.

Before running the actual script to push the data on AWS IoT core, ensure that all the libraries are installed on your local machine.

- Install AWS IoT SDK library on your local machine using pip command as follows.
pip install AWSIoTPythonSDK
pip install awsiotsdk
- There are some other libraries such as sched, random, time etc. are also used. Please make sure that these libraries are also installed.

Once you have installed all the necessary libraries, execute the Bedside Monitor (Device Simulator).

1. Open command prompt and navigate to the location where you have placed BedSideMonitor.py script file

Important Note –

Before running BedSideMonitor.py script please ensure to change the Thing name in BedSideMonitor.py file with the Thing name you created in AWS IoT Core.
(Statement - message['deviceid'] = 'BSM_G101')

2. Execute the script file through command prompt with the command as shown below-

```
python BedSideMonitor.py -e endpoint -r Root_certificate -c Thing_Certificate -k Thing_private_key -id clientid -t topic_name
```

To execute the above command, we need

- **Endpoint** – On AWS IoT Core go to **Settings** & copy the endpoint and save it for future reference.
- **Root_certificate, Thing_Certificate, Thing_private_key** - Certificates and keys are downloaded in the previous steps.
 - We have 2 Root certificates downloaded. You can choose any one filename.
 - Use Device Certificate filename as Thing_Certificate
 - Use Private key certificate filename for Thing_private_key

Either you can place all these certificate files in the same folder where BedSideMonitor.py file is present or you can use the whole file path.
- **clientid - clientid** can be a normal string, make sure that different clientid is provided to send data or multiple devices
- **topic_name.** You can keep the topic_name that is easy to remember. Let's say iot/test

3. So finally, the command-on-command prompt should look similar to this-





```
python BedSideMonitor.py -e a2qdlk1319yncw-ats.iot.eu-north-1.amazonaws.com -r AmazonRootCA1.pem -c 8c7ece8095925df759fe9f475d4bac15aad697bd58265da379a4b459d3ead04a-certificate.pem.crt -k 8c7ece8095925df759fe9f475d4bac15aad697bd58265da379a4b459d3ead04a-private.pem.key -id clientid -t iot/test
```

```
Command Prompt
C:\Users\rages>python3 C:\AWS\BedSideMonitor.py -e a2qdlk1319yncw-ats.iot.eu-north-1.amazonaws.com -r C:\AWS\AmazonRootCA1.pem -c C:\AWS\8c7ece8095925df759fe9f475d4bac15aad697bd58265da379a4b459d3ead04a-certificate.pem.crt -k C:\AWS\8c7ece8095925df759fe9f475d4bac15aad697bd58265da379a4b459d3ead04a-private.pem.key -id clientid -t iot/test -id client01
```

4. In pycharm parameters would look like this

```
-e a2qdlk1319yncw-ats.iot.eu-north-1.amazonaws.com -r AmazonRootCA1.pem -c 8c7ece8095925df759fe9f475d4bac15aad697bd58265da379a4b459d3ead04a-certificate.pem.crt -k 8c7ece8095925df759fe9f475d4bac15aad697bd58265da379a4b459d3ead04a-private.pem.key -id clientid -t iot/test
```

5. Once you run this program message will start getting printed on the screen

```
Command Prompt
2024-09-05 22:15:43,991 - AWSIoTPythonSDK.core.protocol.internal.clients - DEBUG - This custom event callback is for pub /sub/unsub, removing it after invocation...
2024-09-05 22:15:43,991 - AWSIoTPythonSDK.core.protocol.mqtt_core - INFO - Performing sync publish...
2024-09-05 22:15:43,991 - AWSIoTPythonSDK.core.protocol.internal.clients - DEBUG - Filling in custom puback (QoS>0) event callback...
2024-09-05 22:15:44,023 - AWSIoTPythonSDK.core.protocol.internal.workers - DEBUG - Produced [message] event
2024-09-05 22:15:44,023 - AWSIoTPythonSDK.core.protocol.internal.workers - DEBUG - Dispatching [message] event
Received a new message:
b'{"deviceid": "BSM_G101", "timestamp": "2024-09-05 22:15:43.766411", "datatype": "Temperature", "value": 96.0}'
from topic:
iot/test
-----
2024-09-05 22:15:44,023 - AWSIoTPythonSDK.core.protocol.internal.clients - DEBUG - Invoking custom event callback...
2024-09-05 22:15:44,228 - AWSIoTPythonSDK.core.protocol.internal.workers - DEBUG - Produced [puback] event
2024-09-05 22:15:44,228 - AWSIoTPythonSDK.core.protocol.internal.workers - DEBUG - Dispatching [puback] event
2024-09-05 22:15:44,228 - AWSIoTPythonSDK.core.protocol.internal.clients - DEBUG - Invoking custom event callback...
2024-09-05 22:15:44,228 - AWSIoTPythonSDK.core.protocol.internal.clients - DEBUG - This custom event callback is for pub /sub/unsub, removing it after invocation...
2024-09-05 22:15:44,228 - AWSIoTPythonSDK.core.protocol.mqtt_core - INFO - Performing sync publish...
2024-09-05 22:15:44,236 - AWSIoTPythonSDK.core.protocol.internal.clients - DEBUG - Filling in custom puback (QoS>0) event callback...
2024-09-05 22:15:44,245 - AWSIoTPythonSDK.core.protocol.internal.workers - DEBUG - Produced [message] event
2024-09-05 22:15:44,245 - AWSIoTPythonSDK.core.protocol.internal.workers - DEBUG - Dispatching [message] event
Received a new message:
b'{"deviceid": "BSM_G101", "timestamp": "2024-09-05 22:15:43.991642", "datatype": "SP02", "value": 88}'
from topic:
iot/test
-----
```

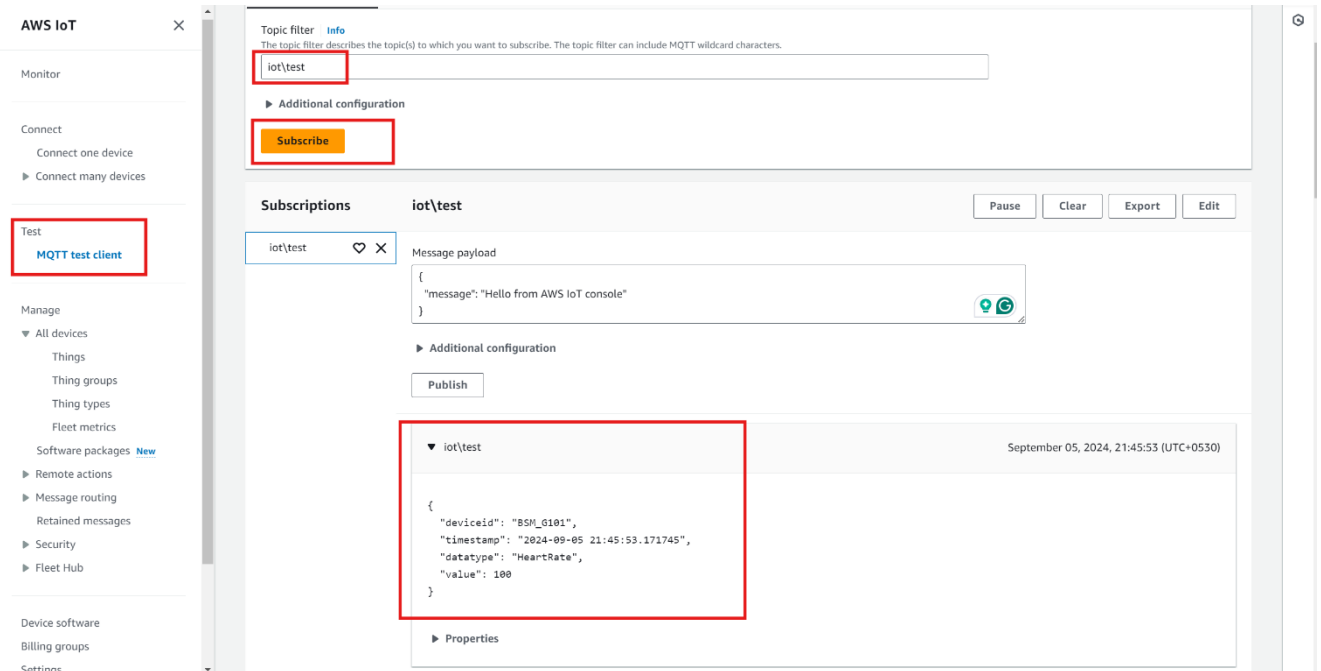
MQTT Test Client: PubSub

In above section we have started pushing messages using device simulator program to AWS IoT Core . Now let's check if we are receiving data from our thing in the IoT core. We can view the data using in AWS IoT core using MQTT test client.

- Click on the Test section and choose MQTT test client.
- Now Enter the topic name which you have provided in the command.
- Click on Subscribe to topic button.



- Once the operation is successful, you can see the data in the same window as below image.



We have successfully executed the script and able to Subscribe the data on MQTT test client. We now need to store the data for the future analysis. We'll do this using AWS Dynamo DB.

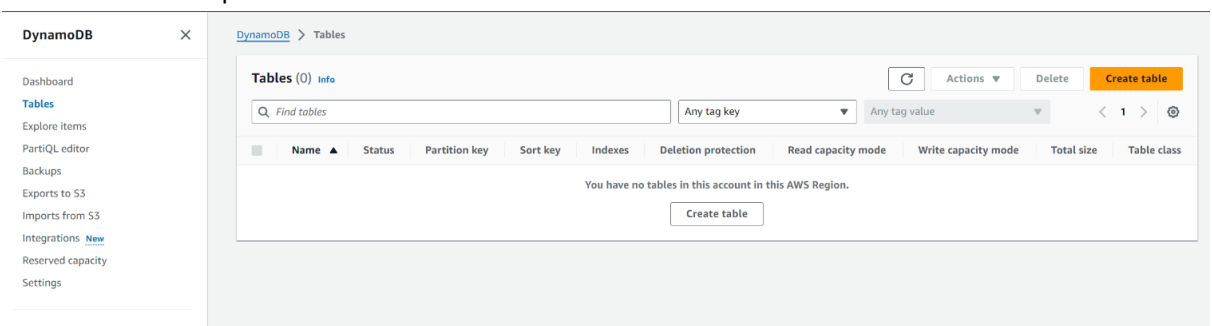
AWS provides a method which enables you to create a table in DynamoDB and create a rule in AWS IoT core to push the data in the DynamoDB table.

You can stop the script execution. Lets move on to the Table and Rule creation for our thing.

AWS DynamoDB : Table Creation

Let's create the table to store the data. Follow the below mentioned steps to create the table.

- From AWS console, search for DynamoDB service in Database service and click on Create table to start the table creation process.



- Provide details such as table_name, partition key and sort key. And then Click on Create table. Here we have provided partition key as deviceid & sort key as timestamp.

Proprietary content. © RKS Tech (www.rkstech.in).

All Rights Reserved. Unauthorized use or distribution prohibited.

Here combination of deviceid & timestamp will become Primary key.

[DynamoDB](#) > [Tables](#) > Create table

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

String ▼

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

String ▼

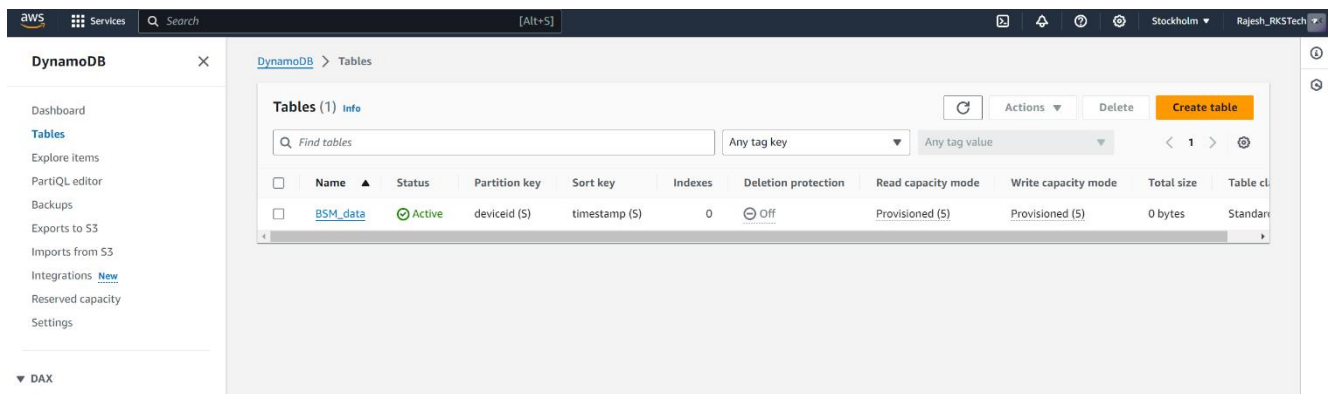
1 to 255 characters and case sensitive.

Table settings

☒ **Default settings**
The fastest way to create your table. You can modify these settings now or after your table has been created.

☐ **Customize settings**
Use these advanced features to make DynamoDB work better for your needs.

3. Successful table creation in DynamoDB will appear as below.



DynamoDB > [Tables](#)

Tables (1) [Info](#)

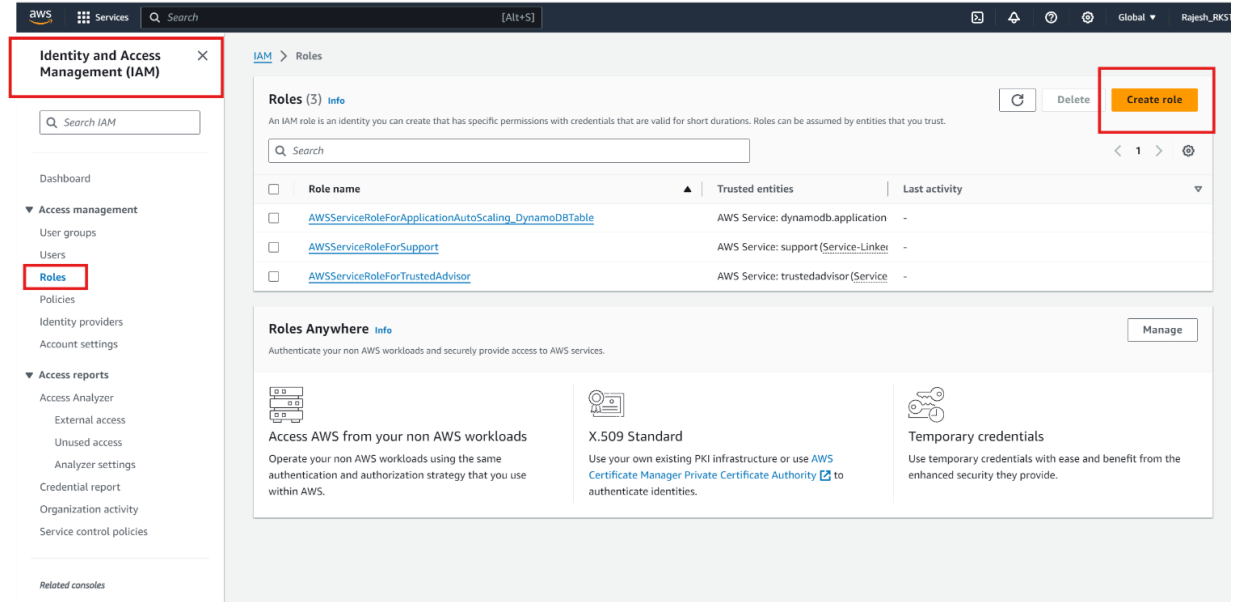
Any tag key Any tag value < 1 >

	Name	Status	Partition key	Sort key	Indexes	Deletion protection	Read capacity mode	Write capacity mode	Total size	Table class
☐	BSM_data	Active	deviceid (S)	timestamp (S)	0	Off	Provisioned (S)	Provisioned (S)	0 bytes	Standard

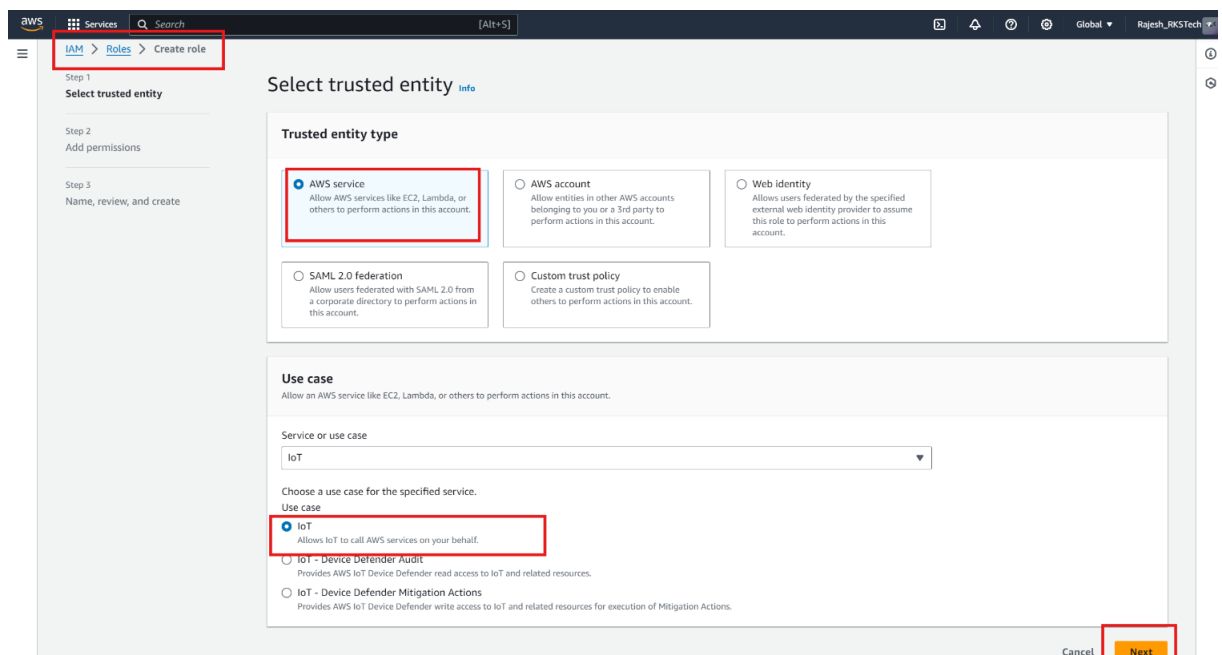
AWS IAM Role Creation

To create a new role follow these steps-

1. On AWS console search for IAM service and click on Roles option. Click on Create role, you will get a screen as follows-



2. Scroll down to choose IOT as the Use Case for other AWS services as shown below-





3. Click Next and you will get the screen for Permission Policies. Click Next again.

IAM > Roles > Create role

Step 1
[Select trusted entity](#)

Step 2
Add permissions

Step 3
Name, review, and create

Add permissions info

Permissions policies (3) info

The type of role that you selected requires the following policy.

Policy name 🔗	Type
AWSIoTLogging	AWS managed
AWSIoTRuleActions	AWS managed
AWSIoTTThingsRegistration	AWS managed

► **Set permissions boundary - optional**

Cancel Previous **Next**

4. Give the Role name and click on Create role. On successful creation you can see the role name list in the roles list. Now go back to the Rule creation step and you can see the role created.

AWS IoT core : Rule Creation

AWS IoT Rules allow automatic action related to upstream AWS services based on data ingested from the devices. There are various rules which can send the data or pre-created information based on the data to various storage services like S3, DynamoDB, Elasticsearch; notification and alarm services like SNS, Cloudwatch metrics/alarms, Kinesis for stream processing, to invoke Lambda to transform the data and send it further, and multiple other analytics and ML AWS services.

You can choose to send all the data coming to a specific set of topics, or filter the data to send based on attributes in the payload. All this is done automatically on-the-fly, reducing the response time for stream processing and alerts/notifications. We'll be looking at setting up a rule to push data to DynamoDB in the next lab

Now we'll create rule to push the IoT core data into the DYNAMODB table we have created in the previous steps.

Follow the below mentioned steps to for the rule creation.

1. Subscribe to the topic name provided in the command used to run the script in previous steps.





- Now go to the Message Routing section and choose Rules. Click on Create and enter name for the rule.

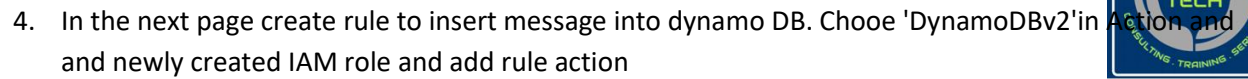
The screenshot shows the AWS IoT console interface. On the left sidebar, under the 'Manage' section, 'Message routing' is expanded, and 'Rules' is highlighted. The main content area shows the 'Rules' page with a 'Create rule' button in the top right corner. The page title is 'Rules (0) Info'. Below the title, there is a search bar and a table with columns: Name, Status, Rule topic, and Created date. The table is empty, and a message states 'No rules. You don't have any rules in eu-north-1.' with a 'Create rule' button below it.

- Provide the query statement as mentioned in the following image. Here enter the topic name (iot/test) given during the script execution.

SELECT * FROM 'topic_name'

The screenshot shows the 'Rule properties' and 'SQL statement' sections of the AWS IoT console. The 'Rule properties' section has a 'Rule description - optional' field with the placeholder text 'A description of your new rule'. The 'SQL statement' section has a 'SQL version' dropdown set to '2016-03-23' and a 'SQL statement' text area containing the query 'SELECT * FROM 'iot/test''.



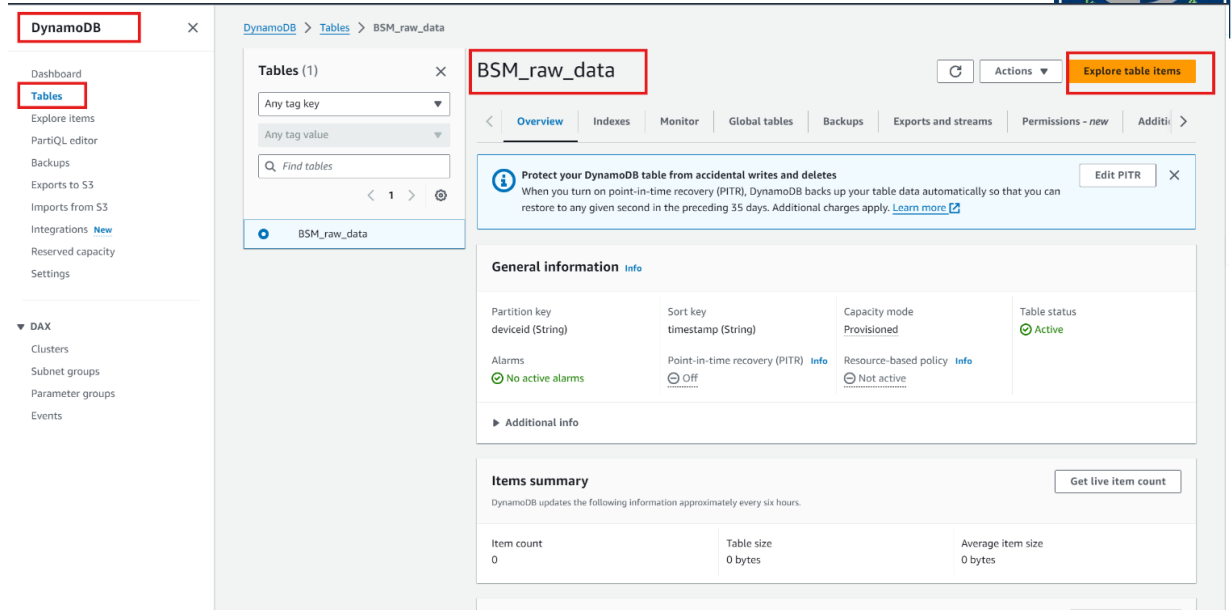


5. On next page provide confirmation to create rule. Successful operation will result into the newly created rule. It will appear similar to the image mentioned below.

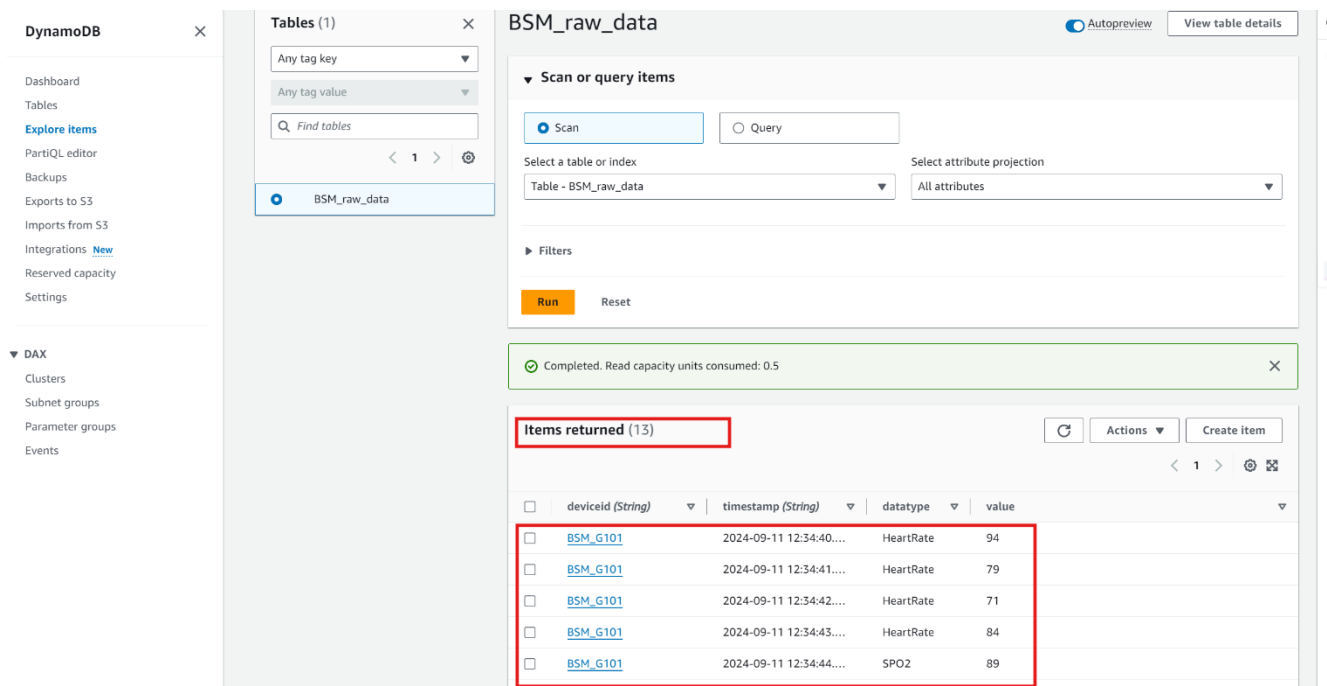
Proprietary content. © RKS Tech (www.rkstech.in).
All Rights Reserved. Unauthorized use or distribution prohibited.



- Go to the DynamoDB page, click on Tables and choose the table created in previous steps.



- Run the command on your local machine which we have used in MQTT test client section. Command will execute the script and data insertion will start. Click on the Explore table items and you can see data getting ingested in the table. It will appear as mentioned in the following image.



deviceid (String)	timestamp (String)	datatype	value
BSM_G101	2024-09-11 12:34:40...	HeartRate	94
BSM_G101	2024-09-11 12:34:41...	HeartRate	79
BSM_G101	2024-09-11 12:34:42...	HeartRate	71
BSM_G101	2024-09-11 12:34:43...	HeartRate	84
BSM_G101	2024-09-11 12:34:44...	SPO2	89



Important

- Once you have completed the hands-on, please ensure to delete all resources created on AWS to avoid any surprise charges.

